SOUTENANCE DE STAGE D'ÉTÉ · 2025/2026

# Détection automatique et autocapture de la CIN tunisienne

Module de capture recto/verso pour Avatar-ID, application Flutter de vérification d'identité à distance

**Mohammed Amine Chaieb**

Codingaholic, La Marsa · encadrant : M. Jendoubi Mohamed Khalil

Université de la Manouba · École Nationale des Sciences de l'Informatique



Format ID-1 · 85,6 × 54 mm · $r_o$ **= 1,586**
les quatre coins que le module doit trouver, seul, dans le flux caméra

# Mon module est la première étape du parcours Avatar-ID : sa qualité conditionne la suite

**CE STAGE**

**SUITE DU PARCOURS AVATAR-ID**

**1**
### Capture de la CIN
détection
qualité d'image
recto puis verso
recadrage, amélioration

→

**2**
### Test de vivacité
la personne est présente, en direct

→

**3**
### Comparaison des photos
photo de la CIN ↔ visage du test de vivacité

→

**4**
### OCR
lecture des champs de la carte

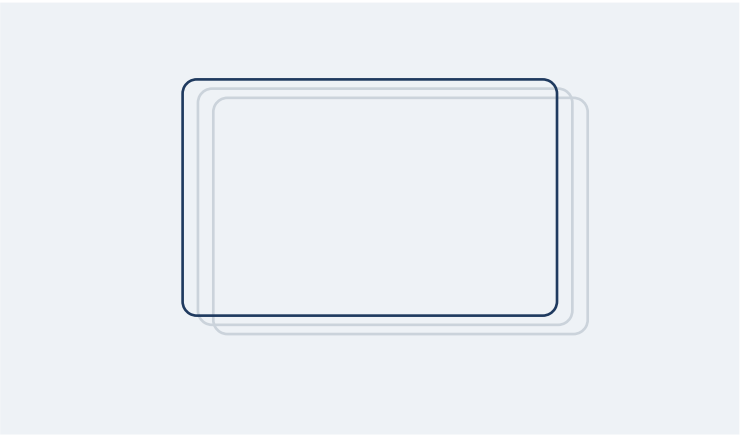
→

**5**
### Enregistrement
sauvegarde du résultat

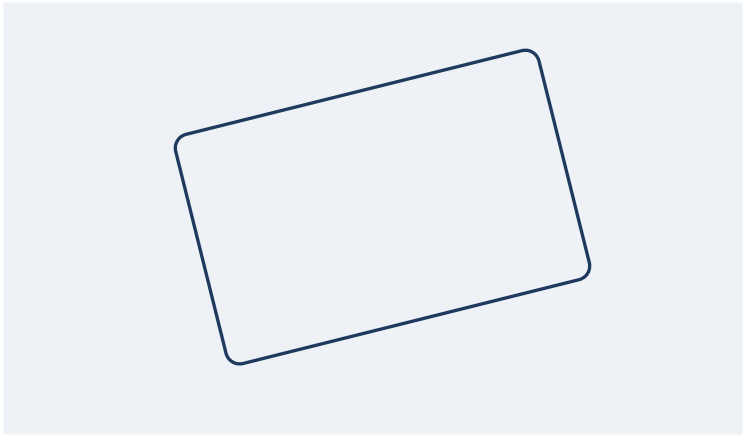
**Ce que le module transmet :** le recto et le verso redressés et orientés, ainsi que la page A4 composée.

**Pourquoi la qualité compte :** une carte nette, bien cadrée et à l'endroit alimente directement la comparaison des photos et l'OCR.

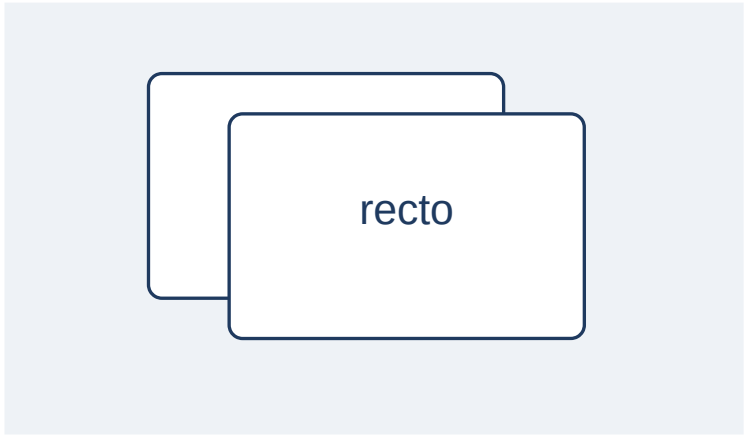
# Une photo libre de CIN peut être floue, de travers ou envoyée deux fois



**Floue**
image inexploitable



**De travers**
carte mal cadrée




**Deux fois le recto**
verso jamais fourni

**Problématique —** comment détecter de façon fiable et en temps réel les contours d'une CIN dans un flux caméra mobile, quels que soient l'éclairage et l'arrière-plan ?

**Sans guidage :** contrôle humain coûteux, nouvelle saisie, et étapes suivantes (photos, OCR) dégradées.

# Cinq fonctions à automatiser, trois cibles chiffrées, deux plateformes

## CE QUE LE MODULE DOIT FAIRE

1. Détecter la carte et ses contours
2. Contrôler netteté, luminosité, stabilité
3. Distinguer recto et verso, dans l'ordre
4. Capturer seul, sans bouton
5. Recadrer et améliorer l'image

## CIBLES DU CAHIER DES CHARGES

**< 100 ms** de traitement par image, idéalement

**≥ 90 %** de recto/verso bien reconnus, en conditions normales

**≈ 2 s** entre carte bien positionnée et capture

**Android et iOS** · un seul code pour les deux

# Seul OpenCV tenait Android + iOS en six semaines, sans jeu de données annoté

| Critère | ML Kit Document Scanner | Modèle entraîné sur mesure | OpenCV (retenu) |
|---|---|---|---|
| **Support iOS** | **Non** | Oui, si entraîné | **Oui** |
| **Jeu de données** | Aucun | **Plusieurs centaines d'images annotées** | **Aucun** |
| **Tenable en 6 semaines** | Oui | **Non** | **Oui** |
| **Effort de réglage** | Faible | Élevé (entraînement) | **Moyen (seuils empiriques)** |

PILE RETENUE



**Flutter**



**Dart**

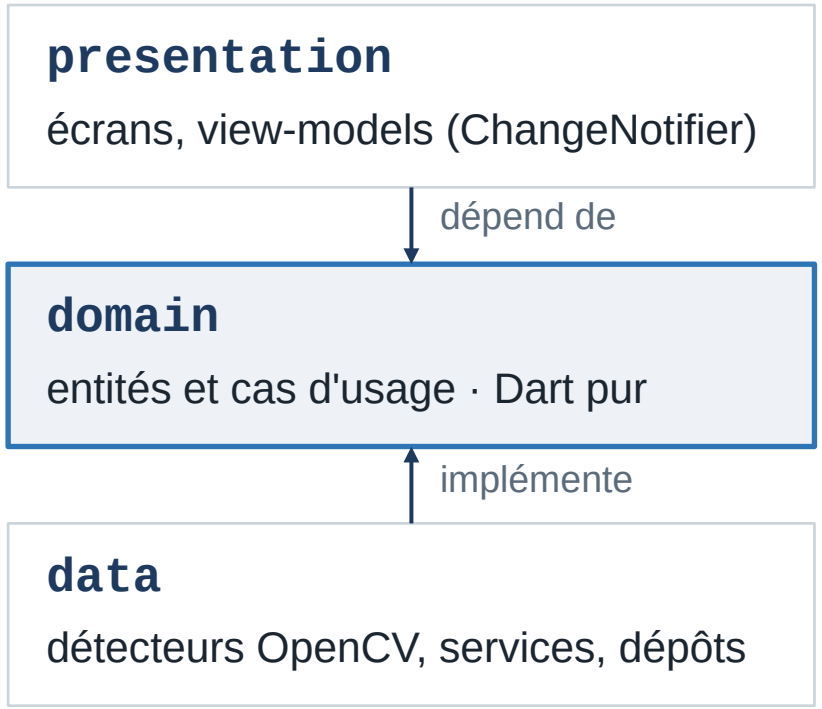


**OpenCV · opencv_dart**

**En orange : le critère qui élimine l'option.** Vision/VisionKit (iOS) écarté aussi : deux chaînes de détection à maintenir.

# Un domaine sans Flutter ni OpenCV rend la logique testable sans caméra

## UN MODULE TYPE (CLEAN ARCHITECTURE)




## SEPT MODULES FEATURE-FIRST : COUCHES RÉELLEMENT PRÉSENTES

| | présentation | domaine | données |
|---|---|---|---|
| `camera_capture` | ● | — | — |
| `document_detection` | ● | ● | ● |
| `image_quality` | — | — | ● |
| `autocapture` | ● | ● | — |
| `image_postprocessing` | ● | ● | ● |
| `result_preview` | ● | ● | — |
| `capture_history` | ● | — | — |

**+ transverses :** `core/services` (caméra, cascade, stockage) et `shared/utils` (conversion d'image, stabilisation)

# Quatre passes de contours et un score géométrique retiennent le quadrilatère de la carte

**1 — Conversion**
frame YUV420 ou BGRA → Mat OpenCV

→

**2 — CLAHE**
contraste local renforcé

→

**3 — 4 passes → masques**
Canny (seuils ×0,66/×1,33 de la médiane) · seuillage adaptatif · saturation HSV + Otsu · gradient morphologique

→

**4 — Score**
0,8 × proportion + 0,2 × taille
retenu si ≥ 0,30
×0,85 si quad approché

→

**5 — Redressement**
homographie → 856 × 540 px
quad resserré de 1,5 %

## CONÇU ET DÉVELOPPÉ PENDANT LE STAGE, AUTOUR DES PRIMITIVES OPENCV

**Sortie anticipée**
les passes suivantes ne tournent que si aucun candidat n'atteint 0,65

**Suivi**
carte trouvée : recherche limitée à ±20 % autour du quad précédent

**Garde-fous**
rejet des quads collés au bord ou couvrant plus de 75 % de l'image

**Isolate Dart**
pipeline hors du thread d'interface ; frame ignorée si l'isolate est occupé

primitive OpenCV — conçu dans le stage (la passe 3 combine les deux)

# Sur fond clair, la détection échouait ; médiane, CLAHE et réglage des seuils l'ont corrigée

## SYMPTÔME

Carte claire sur table blanche : détection souvent en échec, alors qu'elle marchait sur fond sombre.



fond sombre : bord net



fond clair : bord faible

## DEUX CAUSES

**Bug :** seuils de Canny calculés sur la **moyenne** et non sur la médiane : trop hauts dès que la scène est claire.

**Contraste local** trop faible entre la carte et le fond.

## TROIS CORRECTIFS

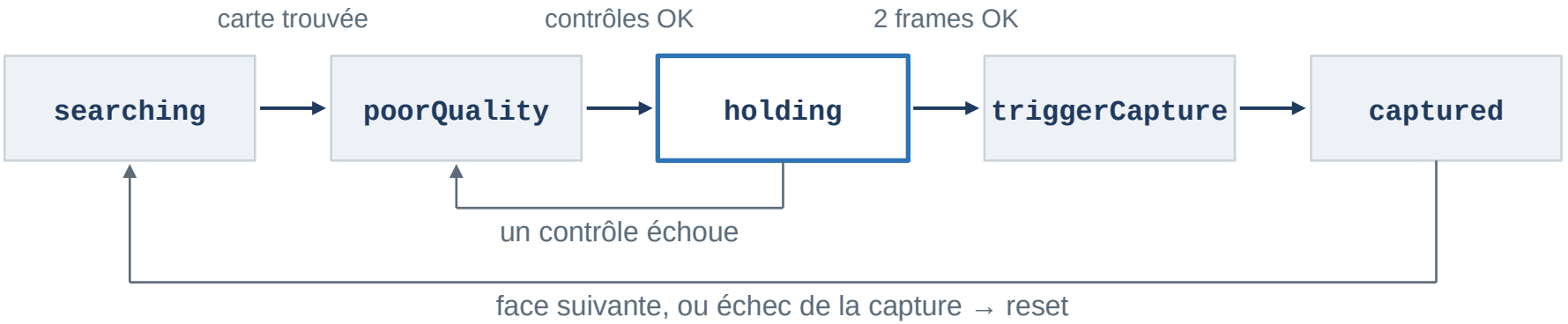
**1** seuils dérivés de la vraie médiane

**2** passe CLAHE avant la détection

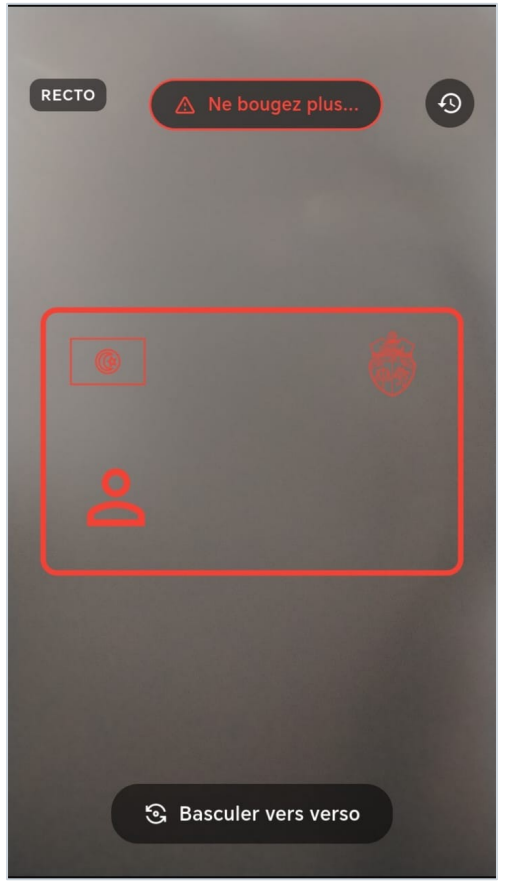
**3** réglage des seuils de score et d'aire du quadrilatère

**Résultat observé :** taux de détection comparable sur fond clair et sur fond sombre.

# La capture part seule après deux frames consécutives nettes, bien exposées et stables




| | |
|---|---|
| **Netteté** | variance du Laplacien, après CLAHE, ≥ 120 (à 856 × 540 px) |
| **Luminosité** | moyenne dans [60 ; 225] et ≤ 12 % de pixels ≥ 250 |
| **Stabilité** | dérive des coins ≤ 2,5 % de la diagonale, sur 3 frames |
| **Contenu** | la face attendue est reconnue (diapositive suivante) |
| **Maintien** | 2 frames consécutives : un compteur, pas une minuterie |




Écran réel : message de qualité « Ne bougez plus… »

# Un visage et deux gabarits identifient le recto ; trois indices votent pour le verso




## RECTO

**Visage :** cascade de Haar standard d'OpenCV, pré-entraînée, testée sur 4 rotations (échelle 1,05, 8 voisins) : elle donne aussi l'orientation.

**Logo + drapeau :** template matching, corrélation ≥ 0,6, CLAHE sur gabarit et région.





gabarits de référence embarqués




## VERSO

**Code-barres :** localisé, non décodé.

**Empreinte :** variance du Laplacien dans [150 ; 4 000].

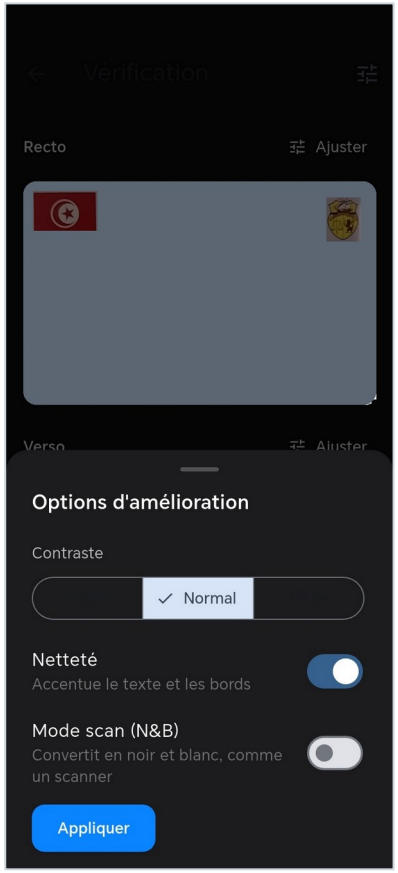
**Ligne de séparation :** Hough probabiliste, bas de la carte.

**Vote** sur 4 rotations : la mieux soutenue gagne.

**Anti-doublon :** après le recto, seule une face qui réunit les trois indices du verso peut déclencher la capture.

# Après les deux captures, l'utilisateur vérifie, ajuste et exporte chaque face

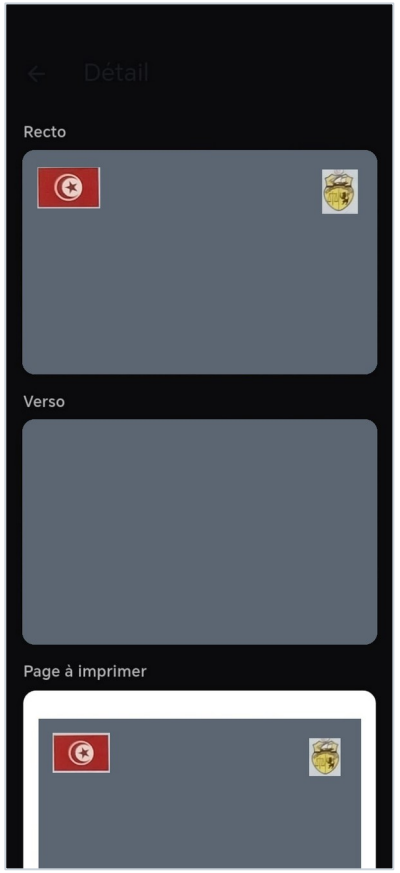
1. **Recto, puis verso**
   capturés seuls, traités sur l'appareil
2. **Vérification**
   contraste, netteté, mode scan (N&B)
3. **Ajustement**
   recadrage par les coins, rotation de 90°
4. **Export**
   galerie : recto, verso et page A4
5. **Historique**
   consultation, partage, suppression




Vérification, amélioration




Recadrage du verso




Page A4 à 300 dpi

# 177 cas de test automatisés passent sans échec, organisés en quatre paliers

**177 / 177**

cas de test réussis · 0 échec
flutter test, en 16 secondes

**Domaine pur → tests rapides :** la séparation en couches (diapositive 6) permet de tester la logique sans caméra ni émulateur.

## QUATRE PALIERS, SELON CE DONT LE TEST A BESOIN

**A** — **Logique pure Dart**
machine à états, décision de capture, stabilité

**B** — **Plateforme simulée (mocktail)**
caméra, permissions, stockage, chargement de la cascade

**C** — **OpenCV réel, images synthétiques**
géométrie du quad, rotation, luminosité, empreinte

**D** — **Cartes réelles, sur téléphone**
visage, code-barres, empreinte, ligne, parcours complet

# Les trois cibles du cahier des charges sont atteintes sur appareil réel

| MESURÉ | CIBLE | CE QUI EST MESURÉ | |
|---|---|---|---|
| **92–100 ms** | < 100 ms par image | temps de traitement d'une frame dans l'isolate, horodaté avant/après | **Atteinte** |
| **92 %** | ≥ 90 %, conditions normales | recto/verso bien reconnus, protocole manuel sur cartes réelles | **Atteinte** |
| **≈ 2 s** | ≈ 2 s | délai chronométré entre le cadre stabilisé et le déclenchement de la capture | **Atteinte** |

**Android et iOS :** le même pipeline OpenCV tourne sur les deux plateformes, avec le même comportement.

# Quatre pistes pour amener le module vers la production

| | |
|---|---|
| **Jeu d'images** | constituer un jeu de CIN plus large pour affiner les seuils et, à terme, ajouter un modèle appris. |
| **Tests réels** | automatiser le palier D avec des images de référence (visage, code-barres, empreinte, ligne). |
| **Timeout** | prévenir l'utilisateur quand aucune carte n'est détectée après un certain délai. |
| **Anti-fraude carte** | analyser la texture pour repérer une carte affichée sur un écran, en amont de la vivacité d'Avatar-ID. |

# Une capture recto/verso autonome, sur Android et iOS, livrée en six semaines

### LIVRÉ

Détection : 4 passes et un score

Autocapture : 4 contrôles, 2 frames

Recto/verso et anti-doublon

Vérification, page A4, export, historique

### MESURÉ

92–100 ms par image

92 % de recto/verso reconnus

≈ 2 s jusqu'à la capture

177 tests réussis · Android et iOS

### PERSPECTIVES

Jeu d'images élargi

Tests réels automatisés

Message de timeout

Anti-fraude sur la carte

**Mon apport :** l'enchaînement des passes et le score du quadrilatère, l'isolate, la machine à états, le vote recto/verso et le post-traitement, autour des primitives d'OpenCV.

**Ce que j'en retiens :** un choix technologique se justifie par les contraintes réelles d'un projet.

# Merci pour votre attention

Je suis à votre disposition pour vos questions.



**Mohammed Amine Chaieb**

Détection automatique et autocapture de la CIN tunisienne · Avatar-ID

# Tous les seuils sont empiriques ; la netteté 120 n'a de sens qu'à 856 × 540 px

| | |
|---|---|
| Score de détection minimal | **0,30** |
| Sortie anticipée | **0,65** |
| Réduction, recherche pleine image | **0,45** |
| Marge de recherche suivie | **20 %** |
| Canny bas / haut (× médiane) | **0,66 / 1,33** |
| Aire de contour minimale | **3,5 %** |
| Aire maximale acceptée | **75 %** |
| Pénalité quad non net | **× 0,85** |
| Proportion / taille (score) | **0,8 / 0,2** |
| Redressement | **856 × 540 px** |

| | |
|---|---|
| Netteté minimale (Laplacien) | **120** |
| Luminosité moyenne | **[60 ; 225]** |
| Pixels saturés tolérés (≥ 250) | **12 %** |
| Dérive de coin maximale | **2,5 % diag.** |
| Frames stables / reset | **3 / 2** |
| Frames de maintien | **2** |
| Haar : échelle / voisins | **1,05 / 8** |
| Corrélation logo / drapeau | **0,6** |
| Variance empreinte | **[150 ; 4 000]** |
| Netteté post-traitement (a) | **0,6** |

# Chaque composant a sa méthode de test, de la logique pure au téléphone réel

| Composant | Méthode de test | Palier |
|---|---|---|
| **Flux caméra** | test sur téléphone réel : ni lag ni coupure | manuel |
| **Détection de la carte** | géométrie pure, images synthétiques, échantillons à différents angles | A · C · D |
| **Netteté et luminosité** | images volontairement floues, sombres ou surexposées | C |
| **Stabilité** | dérive simulée légère, puis franche | A |
| **Recto/verso** | mélange d'images recto et verso | D |
| **Autocapture** | bout en bout : délai et absence de faux déclenchement | A · B · D |
| **Recadrage** | comparaison visuelle : aucun bord coupé | C · D |
| **Anti-doublon** | même face présentée deux fois | D |
| **Gestion des erreurs** | carte cachée, permission caméra refusée | B · D |
| **Test global** | recto puis verso, plusieurs téléphones et éclairages | D |

# La capture est sûre par construction ; la vivacité est vérifiée ensuite par Avatar-ID

## GARANTI PAR CONSTRUCTION

Capture en direct, depuis le flux caméra

Deux faces distinctes, dans l'ordre recto → verso

Qualité contrôlée : netteté, luminosité, stabilité

Traitement sur l'appareil, sans service tiers

## EN AVAL ET PROCHAINES ÉTAPES

Vivacité et comparaison des photos : Avatar-ID

Analyse de texture contre les cartes sur écran

Informer l'utilisateur lors de l'export en galerie

Chiffrement des captures stockées

**Données de test :** les captures montrées dans cette soutenance sont masquées dans les pixels (photo, nom, numéros, code-barres).

# OpenCV fournit les primitives ; l'assemblage, les scores et la logique de capture sont développés

## FOURNI (OPENCV, PLUGINS)

Canny, seuillage adaptatif, Otsu
CLAHE, morphologie
findContours, approxPolyDP
warpPerspective
Laplacien, HoughLinesP
BarcodeDetector
cascade de Haar pré-entraînée
matchTemplate
camera, gal, image

## DÉVELOPPÉ PENDANT LE STAGE

conversion YUV420/BGRA → Mat
DocumentContourDetector : passes, sortie, suivi
CardQuadGeometry : score
DetectionIsolateWorker
StabilityTracker, stabilisateur
machine à états d'autocapture
BackOrientationDetector : vote
CardImageEnhancer, page A4
écrans, historique, export

## RÉFÉRENCES

Canny, IEEE TPAMI, 1986
Otsu, IEEE TSMC, 1979
Zuiderveld, CLAHE, Graphics Gems IV, 1994
Viola & Jones, CVPR, 2001
Matas et al., Hough probabiliste, CVIU, 2000
Hartley & Zisserman, 2004
Martin, Clean Architecture, 2017
ISO/IEC 7810:2019 (ID-1)
Google ML Kit, Document Scanner API [15]